Field Hours

CSCE 715 Secure Authentication Systems

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

A claim is not a session

You separate authentication from authorization on purpose. Authentication is the mechanism that checks a claim of identity. Authorization is the later decision about what that identity may do. A session cookie that appears after a login is not the check itself. It is a handle you issued because a check succeeded. If you skip the check and only look for the cookie's shape, you have mixed up the 2 questions.

You write the claim in one sentence: this subject controls authenticator A, checked at this time, for this relying party. A subject can be a person or a service. You store an account identifier that is stable, not an email address you plan to recycle. When the check fails, you do not open a half session. One failed claim yields 0 privileges. A friendly error string is not a credential.

Two questions you keep apart

Authentication

Checks an identity claim

Uses an authenticator

Failure grants nothing

Authorization

Decides a permitted action

Runs after a real check

Uses the stable account id

A failed claim and a cookie

1. The subject presents authenticator A. Your check fails. You issue no session handle.

2. A caller then shows a cookie-shaped string they invented. You do not treat the shape as a successful claim.

3. Because the claim failed, the authorization step never runs. The subject receives 0 privileges, not a reduced role.

Problem 1. A request carries a well-formed session cookie, but you never ran an authentication check. What have you confused?

Answer. You confused the session handle with the check. Authentication decides whether the claim holds. The cookie is only a handle you issue after success. Shape alone is not a claim.

Problem 2. The authenticator check fails. How many privileges do you grant?

Answer. 0. A failed claim does not open a half session and does not fall through into an authorization decision.

Lecture 2 · 90 minutes

Salt, a slow hash, and a blocklist

You never store the secret a person typed. You store a salt and the output of a slow one-way function. The salt is a random value, unique per account, so two users who pick the same secret do not produce the same stored string. You set a cost parameter, for example 1000 iterations, so one check does far more work than a single hash. A lab unit that spends 1 unit per iteration spends 1000 units on that check.

You also reject secrets you already know are common. A blocklist does that job better than a rule about mixing character classes. Length matters: you accept a long secret, at least 8 characters, and you allow pastes of a phrase. NIST SP 800-63B is the public guide you read for those verifier rules. A pepper, if you add one, is a secret key held outside the account table, not a second salt you store beside the hash.

Store a verifier, not the secret

Draw a unique salt -> Run the slow function -> Store salt and output -> Reject blocklisted secrets

One thousand units per check

1. Account 7 has its own salt. You run 1000 iterations of the slow function on salt plus secret.

2. You store the salt and the output. You do not store the secret. A second account with the same secret and a different salt stores a different output.

3. A guess spends 1000 units, not 1. Four online guesses on this account spend 4000 units before any lockout rule.

Problem 1. One check uses 1000 iterations, and each iteration costs 1 unit. How many units do 4 checks cost?

Answer. 4000. Each check costs 1000 units, and 4 times 1000 is 4000. The salt does not change that cost. It stops identical secrets from sharing one stored output.

Problem 2. Two accounts chose the same secret. Why are the stored outputs different?

Answer. Each account has its own salt, and the slow function takes salt plus secret. Equal secrets with different salts do not produce the same stored string. You still never store the secret itself.

Lecture 3 · 90 minutes

Online guesses meet a lock

You rate-limit checks against one account and against one source. The mechanism is a counter plus a lock. After 5 failed checks you stop accepting new attempts for that account for a window you chose, such as 15 minutes. A success resets the failure counter to 0. The lock is an online control. It does nothing for someone who already stole the verifier file and can guess somewhere else.

You keep the error text dull. The message does not say whether the account exists. You also do not let the lock become a tool that freezes every account a stranger can name: you can require a second signal, such as the source address, before a short lock. Log the failures with the account identifier and the time. A spike of 50 failures on one account in a minute is a page you answer, not a graph you admire.

Five failures, then a lock

1. Count failed checks

2. Lock at five

3. Stay dull in errors

4. Reset only on success

Five failures at one thousand units

1. Each failed check still runs the slow function and costs 1000 units. The counter moves from 0 toward 5.

2. Failure number 5 locks the account for 15 minutes. Online work spent on those failures is 5000 units.

3. A correct secret during the lock does not slip through. When the window ends, one success sets the counter back to 0.

Problem 1. Each check costs 1000 units and you lock at 5 failures. How many units did those failures cost online?

Answer. 5000. Five times 1000 is 5000. The 15-minute lock then stops further online attempts. It does not slow guesses against a stolen verifier file.

Problem 2. Your error text says the account does not exist. What did you give away?

Answer. You told the caller whether the account identifier is valid. The failure message should stay dull and should not split missing accounts from bad secrets.

Lecture 4 · 90 minutes

A second factor bound to the site

You add a second authenticator that is not another copy of the same secret. A time-based code is a common second factor: both sides share a key, and the code is a short truncation of a function of that key and a time step. With a 30-second step and a 6-digit code, the verifier accepts the current step and maybe 1 step of clock skew. A phished code can be replayed until that step ends. You should know that limit.

A phishing-resistant factor binds the check to the site name. The authenticator signs a challenge that includes the origin, and your verifier rejects a signature that names a different origin. A lookalike site can display a prompt and still cannot obtain a signature your real origin will accept. You offer this factor for accounts that can reset other people. Two factors that both live in the same chat app are really 1 basket.

What the second factor adds

A secret the site stores

A time step or a challenge

A check bound to the origin

A thirty-second code versus a signature

1. The account passes the password check. You still want a second factor. The code changes every 30 seconds and shows 6 digits.

2. An impostor site can ask the person to read those digits aloud. Until the step rolls, a forwarded code can pass your verifier.

3. You switch that account to a signature over a challenge that includes your origin. A signature made for another origin fails your check.

Problem 1. A code is 6 digits on a 30-second step, and you allow 1 step of skew. Why is a forwarded code still a problem inside that window?

Answer. The verifier accepts the current step and the skew step. A code read out and reused before those steps end still matches. Binding a signature to the origin closes that replay. A short code does not.

Problem 2. The signed challenge names origin example.test, and your site is example.edu. Do you accept it?

Answer. No. The factor is phishing resistant only if you require the origin in the signature to be your origin. example.test is not example.edu, so this signature fails.

Lecture 5 · 90 minutes

Two error rates for a biometric

A biometric check compares a fresh sample to a stored template. You track two rates. The false accept rate counts impostor matches. The false reject rate counts real subjects turned away. If the false reject rate is 1 in 50, then in 100 genuine tries you expect 2 rejects. If the false accept rate is 1 in 10000, then in 20000 impostor tries you expect 2 false accepts.

You store a template derived from the sample, not a raw image, and you protect it like a verifier. You cannot rotate a fingerprint the way you rotate a password. Revocation means a new template, a new matcher setting, and a second factor so a stolen template is not enough. A threshold that drives false accepts toward 0 pushes false rejects up. You write both rates down before you ship.

The two rates you publish

False accept

Impostor treated as match

1 in 10000 in this lab

Tighten and rejects rise

False reject

Real subject turned away

1 in 50 in this lab

Expect 2 per 100 tries

One hundred genuine tries

1. Your matcher is set so a genuine subject is rejected about 1 time in 50.

2. In a lab of 100 genuine tries, you expect 100 divided by 50, which is 2 false rejects.

3. You record that 2 beside the false accept figure. You do not publish only the rate that looks kinder.

Problem 1. The false reject rate is 1 in 50. How many false rejects do you expect in 250 genuine tries?

Answer. 5. Divide 250 by 50. The false accept rate is a different number and does not enter this count.

Problem 2. The false accept rate is 1 in 10000. How many false accepts do you expect in 20000 impostor tries?

Answer. 2. Divide 20000 by 10000. Lowering this rate further will usually raise false rejects, so you record both rates before you change the threshold.

Lecture 6 · 90 minutes

A challenge, a toy secret, a response

You do not let a client send the same response twice and have it count twice. The mechanism is a challenge. Your server draws a fresh nonce, the client computes a function of the shared secret and that nonce, and you check the result. An old response fails because it was built for a different nonce. On a toy 4-bit secret, you can see the whole shape without pretending the toy is a real key.

The toy is not a fielded system. The secret is the 4 bits 1011, which is 11. The challenge is 0011, which is 3. The response is the exclusive or, 1000, which is 8. You accept 8 only for this challenge. A second login draws a new challenge, so 8 will not match again. A real function is a message authentication code over a long secret. The 4-bit mix only shows why the nonce must change.

One fresh challenge, one check

1. Draw a new nonce

2. Client mixes in the secret

3. Accept only this response

Four bits, one challenge

1. The shared toy secret is 1011. You send the challenge 0011 and you store it for this attempt only.

2. The client returns 1011 xor 0011, which is 1000. In decimal that is 11 xor 3 = 8.

3. You accept 8 for this nonce. Replaying 8 on the next attempt fails, because the next nonce is not

0011.

Problem 1. The toy secret is 1100 (decimal 12) and the challenge is 0101 (decimal 5). What response do you expect under the same exclusive-or rule?

Answer. 1001, which is decimal 9. Bitwise, 1100 xor 0101 is 1001, and 12 xor 5 is 9. You accept it only for this challenge.

Problem 2. The client resends 1001 after you have issued a new challenge. Why do you reject it?

Answer. The response is bound to the nonce that produced it. A new challenge needs a new mix with the secret. The old 1001 is a replay, even though it was correct once.

Lecture 7 · 90 minutes

Compare every byte, then decide

A compare that returns at the first mismatch does different work for different inputs. On a toy token of 4 characters, ab12 versus ax99 can stop at the second character, while ab12 versus ab19 runs further. The time, or a step counter, then tracks the matching prefix. You do not ship that compare for a secret or a token. The leak is the early exit, even when the token is fake.

You compare every position and fold differences into one accumulator. For each of the 4 positions you exclusive-or the bytes and combine that into the accumulator. You look once, after the loop. Equal tokens leave 0. Any difference leaves a nonzero value, and the loop length stays 4. You still test two tokens that differ at index 0 and at index 3.

Two compares of four bytes

Early exit

Stops at first mismatch

Steps track the prefix

Do not use it here

Full fold

Visits all 4 positions

One check at the end

Length does not vary

Tokens ab12 and ab19

1. An early-exit compare of ab12 and ax99 stops at the second character. That is 2 steps.

2. The same style of compare on ab12 and ab19 walks further, because the first 2 characters match. The step count changed.

3. The full fold visits all 4 positions for both pairs and checks one accumulator at the end. Both calls do 4 steps.

Problem 1. You compare a 4-byte token and your loop returns on the first mismatch. Why is that a problem even for a fake token?

Answer. The number of steps tracks how long the prefix matched. A caller who can observe steps learns where the first difference sits. You fold all 4 positions into one accumulator and branch once at the end.

Problem 2. Two tokens differ only at the last byte. How many positions does the full fold visit?

Answer. 4. The loop does not stop early. The accumulator becomes nonzero on the last byte, and you inspect it only after all 4 positions.

Lecture 8 · 90 minutes

An authenticator set you can specify

You can now specify an authenticator set instead of saying the login is secure. You store a unique salt and a slow output, with a cost you can state, such as 1000 units per check. Online, you lock at 5 failures, and those checks cost 5000 units. That lock does not bound guesses on a stolen verifier. You can add a second factor. A 6-digit code on a 30-second step can be forwarded until the step ends.

You can also say when a biometric is fit to ship. You publish both error rates, and you keep a second factor because a template is hard to rotate. A challenge must be fresh: in the 4-bit toy, response 8 belonged to one nonce. Compares of tokens visit every byte. Your spec names the claim, the factors, the lock, and the cost. If the lock is your only control on a stolen table, the spec is not finished.

What the spec must name

Salt and slow cost

Lock after five failures

Second factor and origin

Fresh nonce, full compare

Online cost is not offline cost

1. One check costs 1000 units. The account fails 5 times and locks. Online work is 5000 units.

2. A stolen copy of the verifier is not stopped by that lock. The slow function still costs 1000 units per guess somewhere else, but the counter never runs.

3. You write both sentences in the spec. You add a factor whose signature must name your origin, so a forwarded 6-digit code is not the whole story.

Problem 1. Each check costs 1000 units and the lock trips at 5 failures. What is the online work for those failures, and does the lock cap guesses against a stolen verifier?

Answer. Online work is 5000 units, because 5 times 1000 is 5000. The lock does not cap offline guesses. A stolen verifier can be tested without your counter, which is why the slow function and a second factor still matter.

Problem 2. Your spec uses a 6-digit code with a 30-second step, a 4-byte compare that returns early, and no origin check. Which two controls are missing or weak?

Answer. The compare must visit all 4 bytes before it decides, and the second factor should bind a signature to your origin. A 30-second code can be forwarded inside its step, so the early-exit compare and the unbound code are both unfinished.